



دانشکده مهندسی برق

طراحی و پیاده‌سازی تخصیص گر اولویت
(Priority Arbiter)

بیتا بینائی‌باش

۴۰۱۴۱۱۳۲۷

پروژه نهایی درس مدار منطقی پیشرفته

استاد درس:

آقای دکتر ستار میرزا کوچکی

تیر ۱۴۰۵

فهرست مطالب

۳	هدف پروژه
۴	شرح پروژه
۴	شرح معماری ساختاری
۵	ماژول‌های اصلی
۸	نتایج شبیه‌سازی
۱۰	معماری و منطق پشتیبانی از درخواست‌های اضطراری
۱۱	نتایج شبیه‌سازی معماری داور اولویت با کلاینت اضطراری
۱۳	شرح معماری با کلاینت اضطراری
۱۴	بلوک دیاگرام معماری
۱۵	شرح کد پروژه
۱۷	جمع‌بندی و دستاوردهای پروژه
۱۸	چالش‌ها و پیچیدگی‌های طراحی (مشکالت پیش‌آمده و راهکارها)

پروژه برای تخصیص اولویت بوده است و برای اینکه پروژه کمی پیشرفته تر شود کلاینت اضطراری هم بررسی شده و کد آن زده شده است و کد دو پروژه چون مانند هم هستند در راستای هم توضیح داده شده اند.

➤ هدف پروژه:

هدف اصلی این پروژه، طراحی، توصیف سخت افزاری و پیاده سازی یک سیستم تخصیص گر اولویت ترکیبی (**QoS + Round-Robin Priority Arbiter**) به صورت پارامتریک و ساختاریافته با استفاده از زبان Verilog و اعتبارسنجی آن در محیط ModelSim است.

✓ طراحی مکانیزم کیفیت سرویس (**QoS**): پیاده سازی اولویت مطلق و با تأخیر صفر برای کلاینت شماره ۰ (**VIP**) جهت پاسخگویی آنی به درخواست های حیاتی و وقفه در سیستم.

✓ تخصیص عادلانه پهنای باند (**Round-Robin**): مدیریت چرخشی و هوشمند حق دسترسی به باس (**Bus**) برای کلاینت های معمولی (۱، ۲ و ۳) به منظور جلوگیری از پدیده گرسنگی داده ای (**Starvation**).

✓ پیاده سازی معماری ساختاریافته (**Modular Design**): تفکیک منطق مدار به سه زیرماژول مجزای **priority_logic** (بخش ترکیبی)، **fsm_controller** (بخش کنترل ترتیبی) و **grant_generator** (ثبات و درگاه خروجی) جهت افزایش خوانایی و توسعه پذیری طرح.

✓ حذف خطاهای زمان‌بندی و گلیچ: طراحی به صورت تماماً سنکرون (همگام با کلاک) جهت پایداری سیگنال‌های خروجی gnt در زمان تغییرات همزمان درخواست‌ها بر روی باس.

✓ راستی‌آزمایی عملکردی (Verification): تحلیل دقیق شکل‌موج‌ها و رفتار پویای ماشین حالت محدود (FSM) در شرایط ترافیک سنگین به کمک طراحی یک تست‌بنچ جامع.

➤ شرح پروژه:

در این پروژه، یک تخصیص‌گر اولویت چرخشی (Round-Robin Priority Arbiter) طراحی و پیاده‌سازی شده است.

وظیفه اصلی این مدار، تقسیم عادلانه منابع سیستم (مانند Bus) بین درخواست‌کننده‌های مختلف است.

این مدار به صورت پارامتریک (Parameterized) توسعه یافته است؛ به این معنا که کد نوشته شده برای هر تعداد کلاینت (درخواست‌کننده) قابل گسترش و تنظیم است.

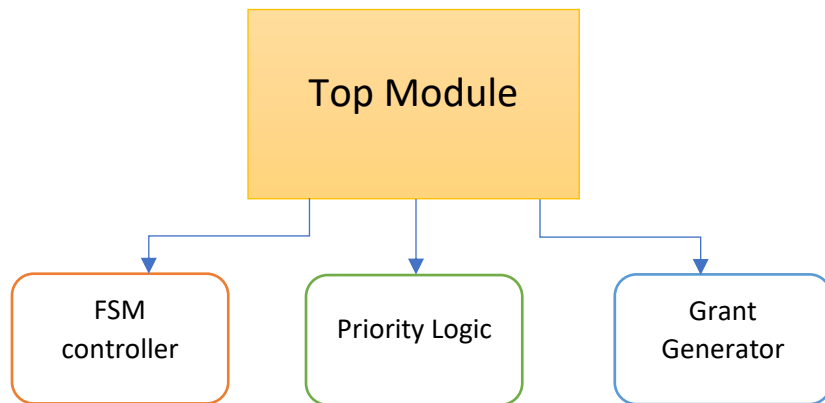
برخلاف یک تخصیص‌گر اولویت ثابت که تنها از منطق ترکیبی ساده بهره می‌برد، طرح حاضر با استفاده از یک ماشین حالت محدود (FSM) وضعیت سیستم را کنترل می‌کند و به مدار قابلیت حافظه‌دار بودن می‌بخشد تا به خاطر بسپارد در سیکل قبلی کدام ماژول به باس دسترسی داشته است. این ویژگی باعث می‌شود سیستم بتواند اولویت‌ها را به صورت هوشمند و چرخشی مدیریت کند تا هیچ کلاینتی در انتظار دائمی نماند.

➤ شرح معماری ساختاری:

طراحی این مدار بر اساس مدل سازی ساختاری (Structural Modeling) و به صورت کاملاً ماژولار انجام شده است.

در این معماری، یک ماژول اصلی (Top Module) وجود دارد که مانند یک چارچوب کلی عمل کرده و سه زیرماژول (Sub-module) مجزا را در دل خود جای داده و از طریق سیم کشی های داخلی به یکدیگر متصل می کند.

این نوع طراحی علاوه بر خوانایی بیشتر کد، عیب یابی و توسعه مدار را بسیار ساده تر کرده و پیچیدگی های منطقی سیستم را به بخش های کوچک تر و قابل مدیریت تقسیم می کند.



شکل ۱. بلوک دیاگرام معماری ساختاری

➤ مازول های اصلی:

سیستم تخصیص گر اولویت چرخشی در این پروژه از سه بلوک یا بخش اصلی تشکیل شده است.

۱. منطق اولویت (Priority Logic):

این بخش یک مدار کاملاً ترکیبی (بدون کلاک) است.

وظیفه آن بررسی درخواست های ورودی (req) و همچنین اشاره گر اولویت (ptr – که نشان میدهد اکنون نوبت کیست) است تا بر اساس قانون اولویت، برنده را مشخص کند.

به عنوان مثال، اگر ptr روی صفر باشد، اولویت به ترتیب با کلاينت های ۰، ۱، ۲ و ۳ است؛ اما اگر ptr روی یک باشد، اولویت شیفت پیدا کرده و از کلاينت ۱ شروع می شود (منظور از کلاينت 0001 = 0، کلاينت 0010 = 1، کلاينت 0100 = 2 و کلاينت 1000 = 3 است).

۲. تولید کننده مجوز (Grant Generator):

این مازول وظیفه تولید سیگنال نهایی خروجی (Grant) برای تخصیص منبع (مانند Bus) به مازول برنده را بر عهده دارد.

عملکرد این بخش مانند یک درگاه یا «دربان» است؛ به این معنا که شماره مازول برنده را از بخش منطق اولویت دریافت می کند، اما سیگنال خروجی (gnt) را تنها زمانی فعال می کند که از سمت ماشین حالت (FSM) سیگنال فعال ساز (enable) را دریافت کرده باشد.

۳. کنترل‌کننده ماشین حالت محدود (FSM controller):

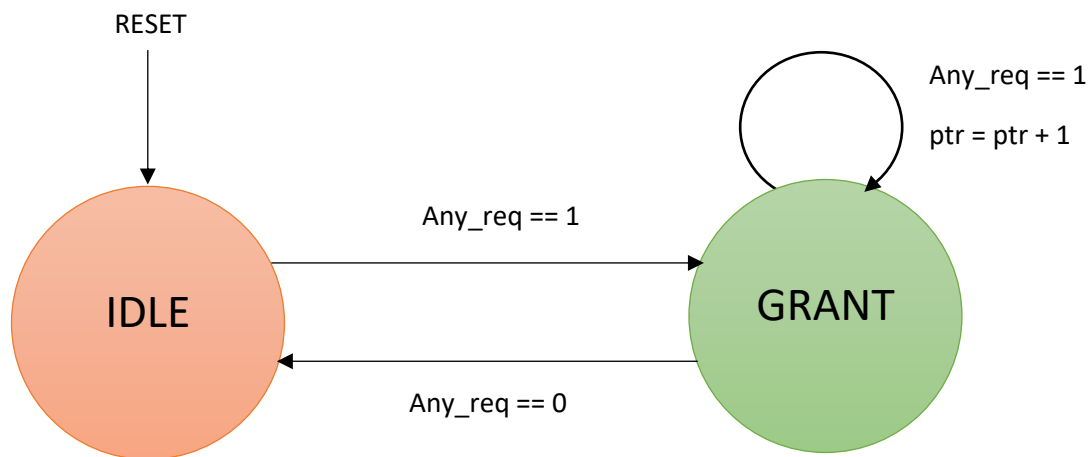
این بخش به عنوان مغز متفکر مدار عمل کرده و به تخصیص‌گر ساده ما، قابلیت حافظه‌دار بودن و چرخش اولویت‌ها را می‌بخشد.

این ماشین حالت دارای دو وضعیت اصلی IDLE (بیکار) و GRANT (در حال سرویس‌دهی) است.

هنگام دریافت درخواست، FSM به حالت GRANT می‌رود، سیگنال enable را برای ماژول تولیدکننده مجوز روشن می‌کند و در نهایت، پس از هر بار سرویس‌دهی، اشاره‌گر نوبت (ptr) را به‌روزرسانی (پلاس پلاس) می‌کند تا عدالت رعایت شده و در کلاک بعدی، اولویت به نفر بعدی برسد.

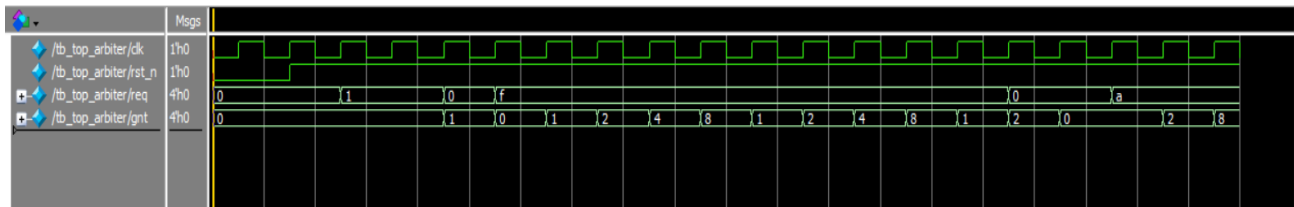
۴. ماژول اصلی (Top Module):

این ماژول، سه بلوک یاد شده را در کنار یکدیگر فراخوانی کرده و پایه‌ها و سیم‌های داخلی را بین آن‌ها متصل می‌کند تا سیستم به صورت یکپارچه عمل کند.

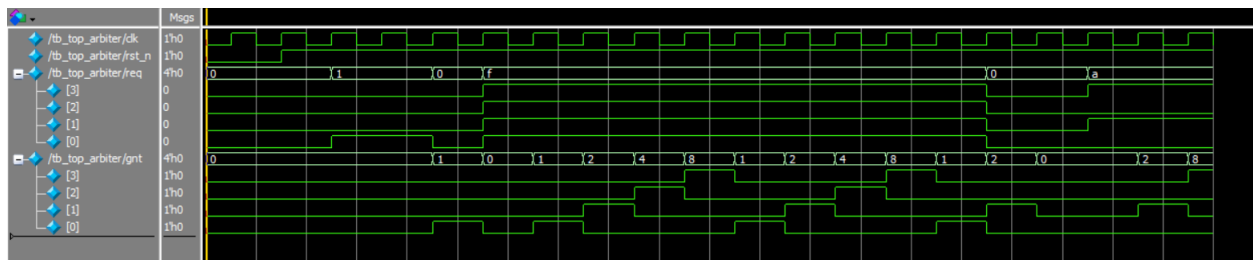


شکل ۲. نمودار انتقال حالت (State Diagram) برای کنترلر ماشین حالت (FSM) در داور چرخشی

➤ نتایج شبیه‌سازی:



شکل ۳. خروجی شبیه‌سازی



شکل ۴. خروجی کامل شبیه‌سازی

برای اثبات صحت عملکرد مدار، یک تست‌بنچ جامع طراحی شد که سناریوهای مختلف ترافیک باس را شبیه‌سازی می‌کند. جهت درک بهتر مقادیر، سیگنال‌ها به صورت One-Hot کدگذاری شده‌اند؛ به این معنا که کلاینت‌های ۰، ۱، ۲ و ۳ در خروجی gnt به ترتیب با مقادیر هگزادسیمال ۱، ۲، ۴ و ۸ (معادل ۰۰۰۱، ۰۰۱۰، ۰۱۰۰، ۱۰۰۰) نمایش داده می‌شوند.

تحلیل زمان‌بندی به شرح زیر است:

حالت آماده‌باش و ریست (۰ تا ۱۵ نانوثانیه): در این بازه، سیگنال rst_n صفر است و مدار کاملاً در حالت آماده‌باش قرار دارد. سیستم به هیچ درخواستی پاسخ نمی‌دهد و خروجی gnt مطلقاً برابر صفر (۰۰۰۰) است.

سرویس‌دهی به درخواست تکی (۲۵ تا ۴۵ نانوثانیه): با اعمال درخواست $req = 0001$ (مربوط به کلاینت ۰)، سیستم درخواستی را دریافت می‌کند. از آنجا که این یک مدار سنکرون است، سیستم پس از یک تاخیر کوتاه (حدود 20 نانوثانیه) پردازش‌های لازم را انجام داده و دقیقاً در لبه بالارونده کلاک بعدی، مجوز دسترسی را به همان کلاینت اختصاص می‌دهد.

($gnt = 0001$) این درخواست روی خط ۴۵ نانوثانیه قطع می‌شود و در همین حین، ماشین حالت (FSM) اشاره‌گر نوبت را به‌روزرسانی می‌کند تا در کلاک‌های بعدی عدالت رعایت شود.

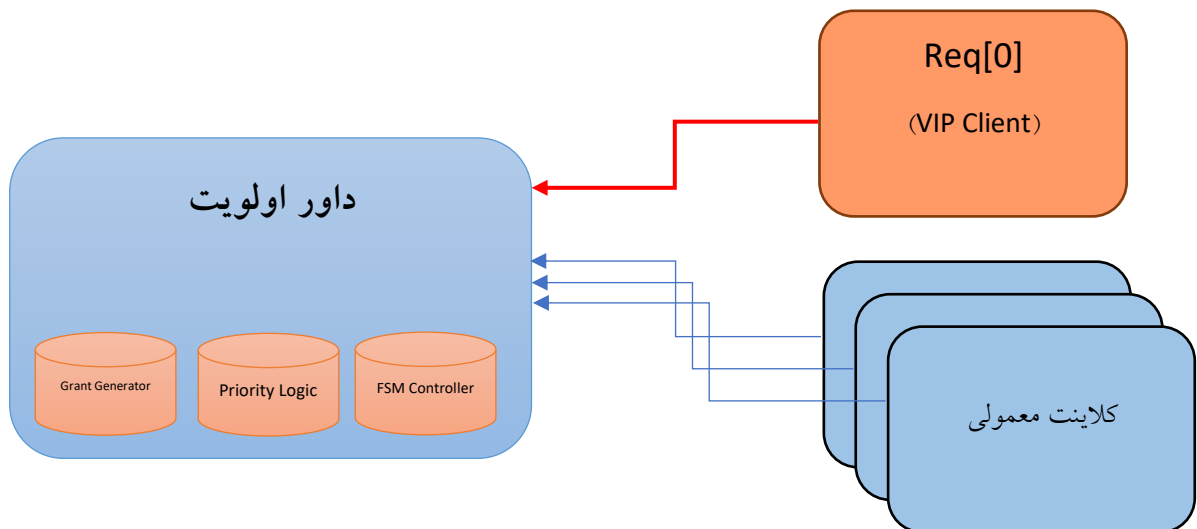
ترافیک سنگین و اثبات حافظه‌دار بودن مدار (۵۵ تا ۱۵۵ نانوثانیه): در زمان ۵۵ نانوثانیه، درخواست همزمان از سوی تمامی کلاینت‌ها صادر می‌شود ($req = 1111$ یا f). در این بخش، ماشین حالت (FSM) هوشمندی خود را نشان می‌دهد؛ از آنجا که باس برای مدتی بیکار (IDLE) بوده است، مدار اولویت را به حالت پیش‌فرض (کلاینت ۱) برمی‌گرداند. سپس خروجی به صورت پله‌ای و چرخشی به ترتیب مقادیر ۲، ۴، ۸ و ۱ را به خود می‌گیرد. این تغییرات چرخشی اثبات می‌کند که سیستم روی یک ماژول قفل نمی‌شود و از پدیده Starvation (انتظار دائمی) جلوگیری می‌کند.

سرویس‌دهی به درخواست‌های پویا (۱۷۵ نانوثانیه به بعد): در این بخش درخواست $req = 1010$ (حرف a در هگزادسیمال) اعمال می‌شود که مختص کلاینت‌های ۱ و ۳ است. خروجی gnt نیز به درستی سایر کلاینت‌ها را نادیده گرفته و صرفاً بین مقادیر ۲ و ۸ نوسان می‌کند.

➤ معماری و منطق پشتیبانی از درخواست‌های اضطراری

معرفی قابلیت تخصیص اولویت اضطراری (VIP Client):

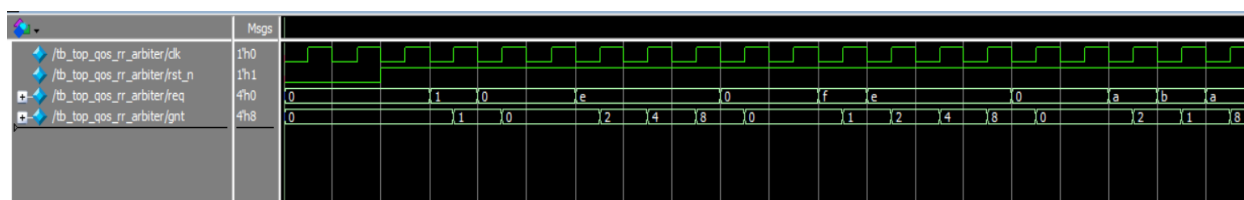
در سیستم‌های دیجیتال واقعی و کنترل‌کننده‌های باس، همواره شرایطی پیش می‌آید که یک ماژول خاص (مانند پورت اعلام خطای سخت‌افزاری یا وقفه‌های بلادرنگ) نباید در صف چرخشی منتظر بماند و نیازمند دریافت خدمات بدون تاخیر است. به همین منظور، معماری این پروژه به گونه‌ای ارتقا یافته است که ترکیبی از دو منطق «اولویت ثابت» و «اولویت چرخشی» باشد. در این معماری، کلاینت شماره ۰ به عنوان کلاینت حیاتی (VIP) با تاخیر صفر، و کلاینت‌های ۱، ۲ و ۳ به عنوان کلاینت‌های معمولی با نوبت چرخشی در نظر گرفته شده‌اند.



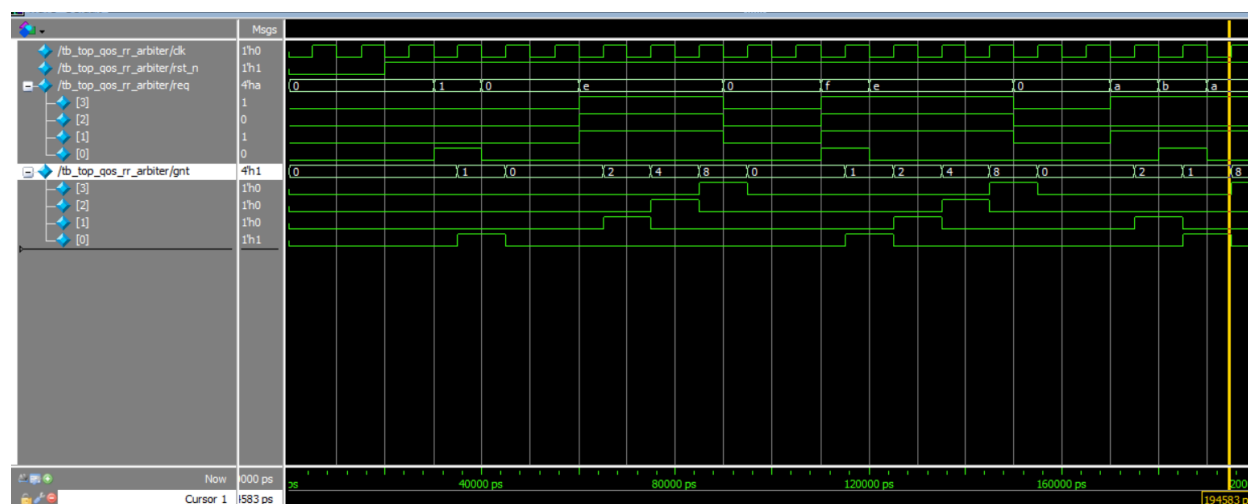
شکل ۵. بلوک دیاگرام معماری پیشرفته

➤ نتایج شبیه‌سازی معماری داور اولویت با کلاینت اضطراری:

پس از کامپایل موفق ماژول‌ها در شبیه‌ساز ModelSim، تست‌بنچ طراحی شده اجرا گردید تا رفتار سیستم یکپارچه تخصیص‌گر اولویت (QoS + Round-Robin) تحت ترافیک‌های ورودی مختلف ارزیابی شود.



شکل ۶. نتایج شبیه‌سازی با کلاینت VIP



شکل ۷. شبیه‌سازی با کلاینت VIP به صورت کامل‌تر

تحلیل زمان‌بندی و رفتار پویای مدار بر اساس نتایج واقعی شبیه‌سازی به شرح زیر است:

موقعیت در زمان ۱۸۰,۰۰۰ پیکوثانیه (180 ns): در این لحظه، باس ورودی درخواست‌ها برابر $req = 1011$ است. با تحلیل بیت‌های این سیگنال، مشخص است که کلاینت شماره ۰ (VIP) درخواست خود را فعال کرده است ($req[0] = 1$).

واکنش مدار: از آنجا که پورت اضطراری فعال است، مدار تخصیص اولویت چرخشی عادی را دور می‌زند. در نتیجه، مجوز نهایی دقیقاً همگام با کلاک به کلاینت اضطراری داده می‌شود (gnt = 0001).

تحلیل تاخیر سنکرون و خروج از حالت اضطراری:

موقعیت در زمان ۱۹۰,۰۰۰ پیکوثانیه (190 ns): در این نقطه زمانی، درخواست کلاینت اضطراری قطع شده و باس ورودی به مقدار req = 1010 تغییر می‌یابد (req[0] = 0). این یعنی کلاینت‌های معمولی ۳ و ۱ درخواست دسترسی دارند.

واکنش مدار و تحلیل تاخیر: با وجود اینکه درخواست VIP قطع شده است، مشاهده می‌شود که در این لبه کلاک خروجی مجوز همچنان روی مقدار قبلی خود یعنی gnt = 0001 باقی مانده است. این رفتار، اثبات مهندسی ماهیت سنکرون مدار و فلیپ‌فلاپ‌های موجود در ماژول grant_generator است؛ به این معنا که تغییرات ترکیبی ورودی، دقیقاً یک سیکل کلاک (تا رسیدن لبه بالارونده بعدی کلاک در زمان ۱۹۵ نانوثانیه) زمان نیاز دارند تا پردازش شده و در خروجی رجیستر شوند. در این بازه، اشاره‌گر راند-رایین کماکان روی مقدار RR Pointer = 2 فریز مانده است.

موقعیت در زمان ۱۹۵,۰۰۰ پیکوثانیه (195 ns): با رسیدن لبه بالارونده بعدی کلاک، سیستم پردازش ورودی req = 1010 را نهایی می‌کند.

واکنش مدار: از آنجا که کلاینت VIP دیگر درخواستی ندارد (req[0] = 0)، ماشین حالت قفل اشاره‌گر را آزاد می‌کند. از طرفی چون اشاره‌گر قبلی روی مقدار ۲ فریز شده بود، مدار ابتدا کلاینت ۲ را بررسی می‌کند؛ اما از آنجا که کلاینت ۲ درخواستی ندارد (req[2] = 0) اولویت بلافاصله به کلاینت بعدی یعنی کلاینت شماره ۳ که درخواستش فعال است (req[3] = 1) منتقل می‌شود.

تغییرات خروجی: در نتیجه، سیگنال مجوز به درستی روی مقدار $gnt = 1000$ (صدور مجوز برای کلاینت ۳) تنظیم شده و اشاره گر نوبت نیز یک واحد به جلو شیفت پیدا می کند تا در چرخه بعدی نوبت را به کلاینت ۱ بدهد ($RR\ Pointer = 3$).

➤ شرح معماری با کلاینت اضطراری:

طراحی این سیستم به صورت یکپارچه و ساختاری انجام شده است. ماژول اصلی (Top Module) شامل سه زیربخش کلیدی است:

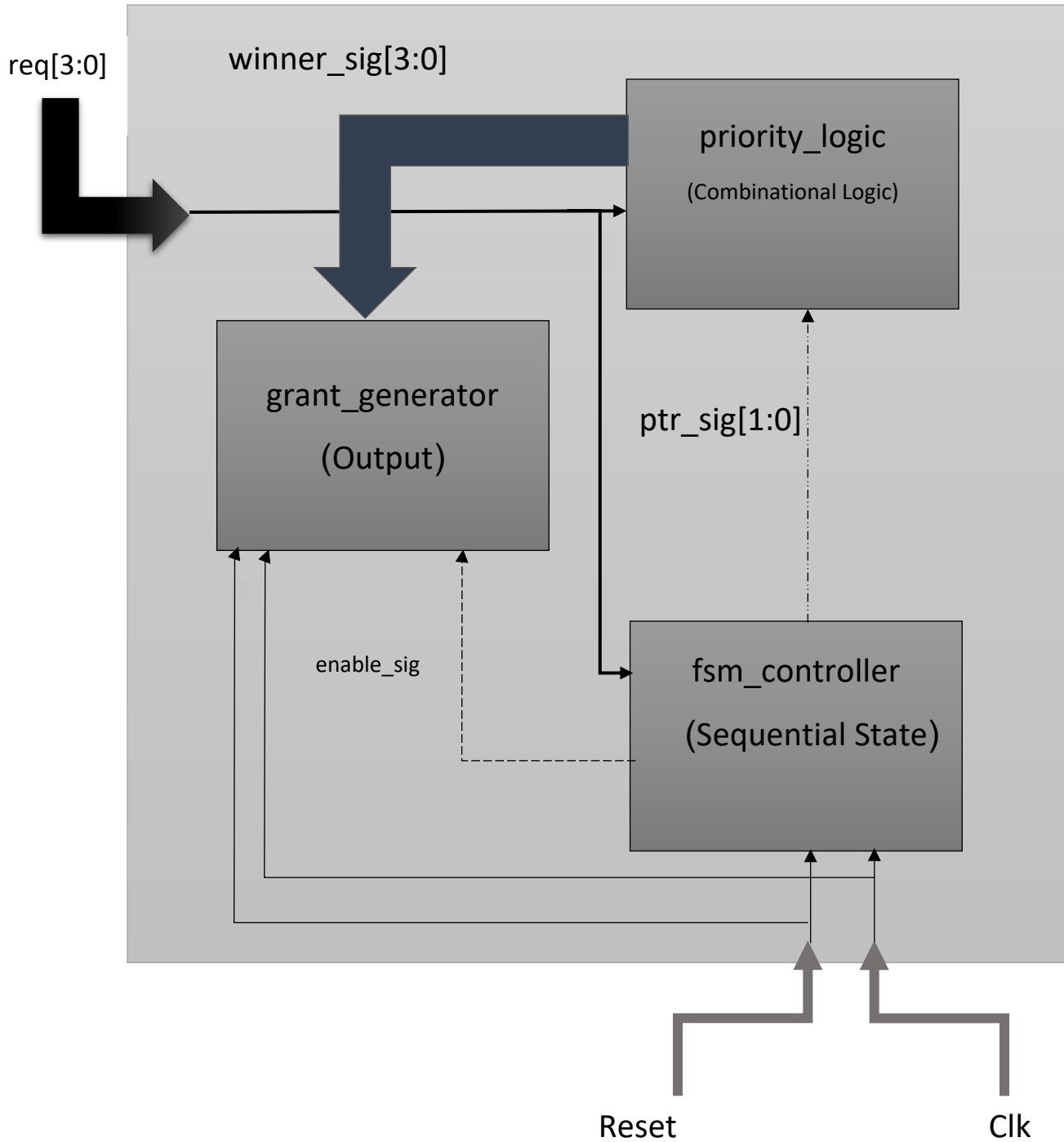
منطق اولویت (Priority Logic): این بخش به عنوان مغز ترکیباتی سیستم عمل کرده و دارای یک معماری دومارحله ای است. در هر کلاک ساینکل، ابتدا وضعیت کلاینت حیاتی بررسی می شود و تنها در صورت بیکار بودن آن، نوبت به صورت چرخشی بین کلاینت های معمولی تقسیم می گردد.

ماشین حالت محدود (FSM Controller): وظیفه اصلی این کنترلر، مدیریت وضعیت سیستم و به خصوص محافظت از نوبت کلاینت های معمولی در زمان ورود درخواست های اضطراری است.

تولیدکننده مجوز (Grant Generator): مانند یک درگاه (فلیپ فلاپ D) عمل کرده و با دریافت سیگنال فعال ساز از FSM، مجوز نهایی را صادر می کند.

➤ بلوک دیاگرام معماری:

Top Module



➤ شرح کد پروژه:

۱. ماژول منطق اولویت (priority_logic):

ساختار ترکیبی این ماژول تماماً درون یک بلوک **always** نوشته شده است تا به عنوان یک مدار ترکیبی خالص، بدون وابستگی به کلاک و در همان لحظه برنده را تعیین کند.

تخصیص اولویت VIP: در اولین سطح از منطق، دستور **if (req[0] == 1'b1)** بررسی می‌شود. این خط کد نشان می‌دهد که کلاينت ۰ بالاترین اولویت را دارد و در صورت فعال بودن، مدار فوراً خروجی **winner = 4'b0001** را تولید کرده و از بررسی سایر شرطها عبور می‌کند.

منطق چرخشی (Round-Robin): در صورتی که VIP درخواستی نداشته باشد، برنامه وارد بدنه **else if (req[3:1] != 3'b000)** می‌شود. در اینجا از یک ساختار **case (ptr)** استفاده شده است تا بر اساس مقدار اشاره گر FSM تصمیم‌گیری شود. به عنوان مثال، اگر **ptr == 2'd1** باشد، مدار با دستور **if(req[1]) winner = 4'b0010** ابتدا نوبت را به کلاينت ۱ می‌دهد و در صورت عدم درخواست، با **else if** های متوالی به سراغ کلاينت‌های ۲ و ۳ می‌رود تا چرخش نوبت‌ها به درستی اعمال شود.

۲. کنترل‌کننده ماشین حالت (fsm_controller):

مدیریت وضعیت‌ها: این بخش بر پایه دو پارامتر **IDLE = 1'b0** و **GRANT = 1'b1** توسعه یافته است. در یک بلوک **always**، اگر هرگونه درخواستی روی باس **req** وجود داشته باشد، متغیر **next_state** برابر **GRANT** قرار گرفته و سیگنال **enable_gnt = 1'b1** فعال می‌شود تا به درگاه خروجی اجازه کار بدهد.

الگوریتم حفظ نوبت (Freeze Pointer): شاه‌کلید اصلی این ماژول در بلوک آپدیت اشاره‌گر (ptr) نهفته است. شرط پیشروی اشاره‌گر به این صورت تعریف شده است:

`if (enable_gnt && !req[0] && req[3:1] != 3'b000) !req[0]` حضور عبارت

در این شرط تضمین می‌کند که اگر سیستم در حال سرویس‌دهی باشد اما کلاینت اضطراری وارد مدار شود، این شرط نقض شده و متغیر ptr آپدیت نمی‌شود (Freeze می‌شود). با این‌که، نوبت کلاینت‌های معمولی پس از اتمام کار VIP محفوظ می‌ماند.

۳. تولیدکننده مجوز (grant_generator):

درگاه سنکرون (Synchronous Gate): این بخش بر خلاف منطق اولویت، با استفاده از یک بلوک ترتیبی `always (posedge clk or negedge rst_n)` پیاده‌سازی شده است. این کار باعث می‌شود تا خروجی نهایی کاملاً همگام با لبه کلاک تغییر کند و از بروز گلیچ (Glitch) در مدار جلوگیری شود.

اعمال سیگنال فعال‌ساز: منطق کنترل خروجی بسیار سراسر است؛ در بدنه این ماژول، سیگنال gnt تنها زمانی مقدار winner را به خود می‌گیرد که شرط `if(enable)` که از سمت FSM می‌آید برقرار باشد. در شرایط بیکاری سیستم یا زمان اعمال ریست (`!rst_n`)، دستور `gnt <= 4'b0000` اجرا شده و تمام مجوزها لغو می‌شوند.

۴. ماژول اصلی (top_qos_rr_arbiter):

ارتباطات داخلی (سیم‌کشی): در این ماژول از هیچ ساختار کنترلی (`if` یا `always`) استفاده نشده است. ارتباطات بین زیرماژول‌ها منحصراً از طریق تعریف سیم‌های میانی نظیر `wire`

winner_sig [3:0] (برای انتقال برنده) و ptr_sig [1:0] wire (برای انتقال اشاره گر نوبت) برقرار شده است.

نمونه سازی ساختاری (Instantiation): این فایل نقش چارچوب اصلی را دارد که در آن سه ماژول قبلی با نام های مستعار u_priority ، u_grant و u_fsm فراخوانی شده اند و پورت های ورودی/خروجی آنها به سیم های داخلی و پورت های اصلی مدار گره خورده اند تا سیستم به یک بلوک واحد و یکپارچه تبدیل شود.

➤ جمع بندی و دستاوردهای پروژه:

در این پروژه، یک سیستم تخصیص گر اولویت ترکیبی (QoS + Round-Robin) با موفقیت کامل به زبان سخت افزاری وریلاگ (Verilog) طراحی و پیاده سازی شد. اهداف و دستاوردهای کلیدی این طرح به شرح زیر است:

* معماری ساختاریافته و ماژولار: با الهام از استراتژی های طراحی مدارهای منطقی، سیستم به سه زیرماژول اصلی (منطق اولویت، درگاه خروجی و کنترلر ماشین حالت) تقسیم شد که خوانایی، عیب یابی و توسعه پذیری مدار را به شدت افزایش داد.

* مدیریت هوشمند ترافیک: ترکیب منطق اولویت مطلق (برای کلاینت شماره ۰ یا VIP) با منطق اولویت چرخشی (برای کلاینت های معمولی ۱، ۲ و ۳)، باعث شد مدار بتواند از گرسنگی (Starvation) کلاینت های معمولی جلوگیری کرده و در عین حال در شرایط اضطراری با کمترین تاخیر واکنش نشان دهد.

➤ چالش‌ها و پیچیدگی‌های طراحی (مشکلات پیش‌آمده و راهکارها):

در طول مسیر توسعه معماری و کدنویسی این مدار، با چندین چالش تحلیلی و سخت‌افزاری مواجه شدیم که با اتخاذ استراتژی‌های مناسب برطرف گردیدند:

* **تداخل اولویت‌ها و حفظ حافظه سیستم (Pointer Freezing):** یکی از بزرگترین پیچیدگی‌های منطقی سیستم زمانی رخ می‌داد که مدار در حال چرخش نوبت بین کلاینت‌های معمولی بود و ناگهان یک درخواست اضطراری (VIP) صادر می‌شد. چالش این بود که ماشین حالت (FSM) نباید نوبت کلاینت‌های معمولی را فراموش می‌کرد. این مشکل با طراحی یک مکانیزم هوشمند در بلوک به‌روزرسانی FSM (تعریف شرط `!req[0]`) برطرف شد؛ به گونه‌ای که با ورود کلاینت اضطراری، اشاره‌گر راند-رابین در جای خود فریز (Freeze) شده و پس از رفع اضطرار، کار خود را ادامه می‌دهد.

* **جلوگیری از تولید گلیچ (Glitch) و شرایط رقابتی:** در طراحی‌های ترکیبی خالص، تغییرات همزمان چند ورودی ممکن است باعث ایجاد پالس‌های ناخواسته (Glitch) و شرایط رقابتی (Race Condition) در خروجی شود. برای حل این چالش، معماری مدار به صورت دو مرحله‌ای ارتقا یافت و بلوک `grant_generator` به عنوان یک درگاه سنکرون (ثبات خروجی همگام با کلاک) به سیستم اضافه شد. این کار باعث گردید خروجی نهایی کاملاً تمیز، پایدار و صرفاً در لبه‌های بالارونده کلاک تغییر کند.

* **زمان‌بندی و مدیریت تاخیرهای سنکرون (Synchronous Delays):** هماهنگ‌سازی ماشین حالت ترتیبی با منطق اولویت ترکیبی، نیازمند دقت بالایی در تحلیل زمان‌بندی بود. یکی از مشکلات اولیه این بود که بین صدور درخواست و تخصیص مجوز تداخل زمانی

ایجاد می‌شد. با تفکیک دقیق سیگنال‌های ترکیبی (مانند `next_state`) از سیگنال‌های ترتیبی (مانند `state`) و اعمال تاخیرهای منطقی یک سیکل کلاک، پایداری زمان‌بندی مدار در شرایط ترافیک سنگین (درخواست همزمان تمام کلاینت‌ها) به طور کامل تضمین شد.